Lecture 3. Data Wrangling

dplyr verbs, grouping and summaries, joins, the Uber example

Contents

3.1 A grammar of data wrangling **2**

3.1.1 The single-table verbs 2

3.2 Relational data **4**

3.3 Example: Uber pickups in New York **5**

3.4 Selected exercises **6**

3.1 A grammar of data wrangling

Motivation. ggplot2 gave a grammar for graphics; dplyr gives one for manipulating data. A handful of verbs, each doing one thing and each taking and returning a data frame, can be chained to answer most questions about a table.

`dplyr` was designed to provide the common data-manipulation tools, to be fast on in-memory data, and to separate the operations from the data source (the same code works on a database). It works on data frames and on **tibbles**, trimmed-down data frames (class `tbl_df`) that print only the first rows and the columns that fit, with their types. The introductory vignette is `vignette("dplyr", package = "dplyr")`.

We use `ggplot2::presidential`:

```
presidential
```

```
## # A tibble: 12 x 4
##   name       start      end        party
##   <chr>      <date>     <date>     <chr>
## 1 Eisenhower 1953-01-20 1961-01-20 Republican
## 2 Kennedy    1961-01-20 1963-11-22 Democratic
## 3 Johnson    1963-11-22 1969-01-20 Democratic
## 4 Nixon      1969-01-20 1974-08-09 Republican
## 5 Ford       1974-08-09 1977-01-20 Republican
## # i 7 more rows
```

```
class(presidential)
```

```
## [1] "tbl_df"     "tbl"        "data.frame"
```

3.1.1 The single-table verbs

- `filter()` selects rows; `select()` selects columns;
- `mutate()` adds columns; `arrange()` reorders rows;
- `summarize()` collapses a data frame to a single row, usually together with `group_by()`.

filter(). The first argument is the data; the second is a logical vector, and only rows where it is `TRUE` are kept. Comparisons use `>`, `>=`, `<`, `<=`, `!=`, `==`; set membership `x %in% Y`; and they combine with `&`, `|`, `!`.

```
x <- c(2, 1, 3, 0)
x == 0 | x == 1
```

```
## [1] FALSE  TRUE FALSE  TRUE
```

```
x %in% c(0, 1)
```

```
## [1] FALSE  TRUE FALSE  TRUE
```

```
filter(presidential, party == "Republican")
```

```
## # A tibble: 7 x 4
##   name       start      end        party
##   <chr>      <date>     <date>     <chr>
## 1 Eisenhower 1953-01-20 1961-01-20 Republican
## 2 Nixon      1969-01-20 1974-08-09 Republican
## 3 Ford       1974-08-09 1977-01-20 Republican
## 4 Reagan     1981-01-20 1989-01-20 Republican
## 5 Bush       1989-01-20 1993-01-20 Republican
## # i 2 more rows
```

Missing values are contagious: `NA > 5`, `NA + 10` and even `NA == NA` are `NA`. Test with `is.na()`; to drop observations with a missing variable, `filter(!is.na(variable))`.

select() takes the data and a list of columns. For wide data the helpers `starts_with()`, `ends_with()`, `contains()`, `num_range()` select by name pattern:

```
flights %>% select(ends_with("time"))
```

```
## # A tibble: 336,776 x 5
##   dep_time sched_dep_time arr_time sched_arr_time air_time
##      <int>          <int>    <int>          <int>    <dbl>
## 1      517            515      830            819      227
## 2      533            529      850            830      227
## 3      542            540      923            850      160
## 4      544            545     1004           1022      183
## 5      554            600      812            837      116
## # i 336,771 more rows
```

Combining verbs and the pipe. Which Democratic presidents served since Watergate? Nesting `select(filter(presidential, start > 1973 & party == "Democratic"), name)` works but reads inside out. The pipe `%>%` (keyboard shortcut Ctrl+Shift+M) passes the left side as the first argument, so `df %>% filter(cond)` equals `filter(df, cond)`:

```
presidential %>%
  filter(start > 1973 & party == "Democratic") %>%
  select(name)
```

```
## # A tibble: 3 x 1
##   name
##   <chr>
## 1 Carter
## 2 Clinton
## 3 Obama
```

Read it as "take `presidential`, then keep the Democrats who started after 1973, then select the name".

mutate() creates variables. The length of each term in years, using lubridate's intervals and durations:

```
my_presidents <- presidential %>%
  mutate(term_length = interval(start, end) / dyears(1))
```

It is good style to save the result as a new object rather than overwrite data from an external source.

arrange() sorts a data frame (`sort()` sorts only vectors); ties in the first key are broken by later keys:

```
my_presidents %>% arrange(desc(term_length), party)
```

```
## # A tibble: 12 x 5
##   name       start      end        party      term_length
##   <chr>      <date>     <date>     <chr>            <dbl>
## 1 Clinton    1993-01-20 2001-01-20 Democratic           8
## 2 Obama      2009-01-20 2017-01-20 Democratic           8
## 3 Eisenhower 1953-01-20 1961-01-20 Republican           8
## 4 Reagan     1981-01-20 1989-01-20 Republican           8
## 5 Bush       2001-01-20 2009-01-20 Republican           8
## # i 7 more rows
```

summarize() and group_by(). Alone, `summarize()` reduces each column to one value; `n()` counts rows and `sum(party == "Democratic")` counts the rows satisfying a condition, since `TRUE` counts as 1.

```
my_presidents %>%
  summarize(N = n(), first_year = min(year(start)), last_year = max(year(end)),
```

```
            num_dems = sum(party == "Democratic"),
            years = sum(term_length), avg_term_length = mean(term_length))
```

```
## # A tibble: 1 x 6
##       N first_year last_year num_dems years avg_term_length
##   <int>      <dbl>     <dbl>    <int> <dbl>           <dbl>
## 1    12       1953      2021        5    68            5.67
```

Units [Supplement]

The slides compute `years = sum(term_length) / 365.25` and obtain 0.186. Since `term_length` is already measured in years, the division is not needed: the twelve presidents together served about 68 years (1953–2021).

Grouping makes the same computations run for each party separately, which turns the summary into a comparison:

```
my_presidents %>%
  group_by(party) %>%
  summarize(N = n(), avg_term_length = mean(term_length),
            std_term_length = sd(term_length))
```

```
## # A tibble: 2 x 4
##   party          N avg_term_length std_term_length
##   <chr>      <int>           <dbl>           <dbl>
## 1 Democratic     5            5.6             2.34
## 2 Republican     7            5.71            2.32
```

Average terms hardly differ between the parties.

3.2 Relational data

Motivation. A database stores different kinds of things in different tables and links them by keys, both to save space and to keep each fact in one place. Analyses then need to bring the tables back together.

Multiple linked tables are called **relational data**, because the relations, not only the tables, matter. dplyr has three families of two-table verbs: *mutating joins* (add variables from matching rows of another table), *filtering joins* (keep or drop rows depending on whether they match) and *set operations* (treat rows as set elements). All assume tidy tables.

In `nycflights13`, storing "American Airlines Inc." in each of 336 776 rows of `flights` would waste space and invite inconsistencies, so `flights` stores the two-letter code and `airlines` maps codes to names. The keys: `flights` connects to `airlines` by `carrier`, to `planes` by `tailnum`, to `airports` by `origin` and `dest`, and to `weather` by `origin` and `year`, `month`, `day`, `hour`.

inner_join() keeps rows with a match in both tables:

```
flights_joined <- flights %>%
  inner_join(airlines, by = c("carrier" = "carrier"))
flights_joined %>% select(carrier, name, flight, origin, dest)
```

```
## # A tibble: 336,776 x 5
##   carrier name                  flight origin dest
##   <chr>   <chr>                  <int> <chr>  <chr>
## 1 UA      United Air Lines Inc.   1545 EWR    IAH
## 2 UA      United Air Lines Inc.   1714 LGA    IAH
## 3 AA      American Airlines Inc.  1141 JFK    MIA
## 4 B6      JetBlue Airways          725 JFK    BQN
## 5 DL      Delta Air Lines Inc.     461 LGA    ATL
## # i 336,771 more rows
```

left_join() keeps all rows of the first table, filling `NA` where there is no match. Restrict airports to the Pacific time zone and compare:

```
airports_pt <- airports %>% filter(tz == -8)
flights %>% inner_join(airports_pt, by = c("dest" = "faa")) %>% nrow()
```

```
## [1] 46324
```

```
flights %>%
  left_join(airports_pt, by = c("dest" = "faa")) %>%
  summarize(num_flights = n(),
            num_flights_pt = sum(!is.na(name)),
            num_flights_not_pt = sum(is.na(name)))
```

```
## # A tibble: 1 x 3
##   num_flights num_flights_pt num_flights_not_pt
##         <int>          <int>              <int>
## 1      336776          46324             290452
```

The inner join keeps only the 46 324 west-coast flights; the left join keeps all flights and marks the others with a missing airport name. Other joins: `right_join()` keeps all rows of the second table, `full_join()` all rows of both, `semi_join()` the rows of the first table that have a match (without adding columns), and `anti_join()` those that do not.

3.3 Example: Uber pickups in New York

Motivation. A realistic wrangling task: several monthly files, date-times stored as text, and questions about when and where demand occurs. The steps are always the same: combine, parse, derive variables, group, summarise, plot.

The data are about 4.5 million Uber pickups in New York City, April–September 2014, in six monthly CSV files with columns `Date/Time`, `Lat`, `Lon` and `Base` (the company code). The files are not in an R package, so the code is shown without output.

```
files <- sprintf("data/uber-raw-data-%s14.csv", c("apr", "may", "jun", "jul", "aug", "
    sep"))
data_2014 <- map_dfr(files, read_csv) %>%          # stack the six months
  mutate(dt = mdy_hms(`Date/Time`),
         month = month(dt, label = TRUE), day = day(dt),
         dayofweek = wday(dt, label = TRUE), hour = hour(dt))
```

The slides combine the files with `rbind()` and parse with `as.POSIXct()`; `map_dfr()` and lubridate do the same in fewer steps [Supplement]. Each question is then a grouped count followed by a bar chart:

```
data_2014 %>%
  count(hour, name = "Total") %>%
  ggplot(aes(hour, Total)) +
  geom_col(fill = "steelblue") +
  scale_y_continuous(labels = scales::comma) +
  ggtitle("Trips every hour")
data_2014 %>%
  count(month, dayofweek, name = "Total") %>%
  ggplot(aes(month, Total, fill = dayofweek)) +
  geom_col(position = "dodge")
```

Trips by hour, by hour and month, by day of the month, by month and by weekday within month all follow this pattern; `scales::comma` formats large counts. Plotting `Lon` against `Lat` for all pickups, with axis limits around the city (latitude 40.58–40.92, longitude −74.15 to −73.70), draws a map of New York from the data alone, and colouring by `Base` shows the areas served by each company.

A small lookup table of base codes and names can be joined with `left_join(by = c("Base" = "base_code"))` to label them.

Overplotting millions of points [Supplement]

With 4.5 million points, `geom_point()` is slow and saturates. Smaller points with transparency (`size = 0.1, alpha = 0.05`), or 2-d bins (`geom_bin2d()`, `geom_hex()`), show density instead of a solid blob.

3.4 Selected exercises

Motivation. The verbs of this lecture are simple, but three facts about them are worth proving once: how many rows a join returns, why missing values propagate the way they do, and how grouped summaries combine into overall ones.

Exercise 3.1 (size of a join)

Let `x` and `y` be tables joined by a key. For a key value k let a_k and b_k be the numbers of rows of `x` and of `y` with key k (rows with a missing key match nothing).
(a) Show that `inner_join(x, y)` has $\sum_k a_k b_k$ rows and `left_join(x, y)` has $\sum_k a_k \max(b_k, 1)$ rows.
(b) Deduce that `left_join(x, y)` has exactly as many rows as `x` if and only if every key value of `x` occurs at most once in `y`.
(c) Show that `semi_join(x, y)` and `anti_join(x, y)` split the rows of `x` into two disjoint parts, so their row counts add up to that of `x`.

Solution [Supplement].

(a) An inner join outputs one row for every pair (row r of `x`, row s of `y`) with equal keys. Pairs with key k number $a_k b_k$, and pairs with different keys are disjoint, so the total is $\sum_k a_k b_k$. A left join outputs the same pairs, plus one row (filled with `NA`) for each row of `x` that has no partner. For key k this gives $a_k b_k$ rows if $b_k \geq 1$ and a_k rows if $b_k = 0$, that is $a_k \max(b_k, 1)$ in both cases.

(b) The number of rows of `x` is $\sum_k a_k$, so by (a) the difference is $\sum_k a_k \{\max(b_k, 1) - 1\}$. Every term is ≥ 0, so the difference is 0 if and only if $\max(b_k, 1) = 1$, i.e. $b_k \leq 1$, for every k with $a_k > 0$.

(c) A semi join keeps a row of `x` once if its key has $b_k \geq 1$, and an anti join keeps it once if $b_k = 0$. Exactly one of the two conditions holds for each row, so each row of `x` appears in exactly one of the two results, once, without columns from `y`. In particular a semi join never duplicates rows, unlike an inner join when $b_k \geq 2$. ■

Exercise 3.2 (logic of missing values; adapted from R4DS, Exercise 5.2.4.4)

`NA` stands for a value that exists but is unknown. Explain why `NA^0` is 1, `NA | TRUE` is `TRUE` and `FALSE & NA` is `FALSE`, while `NA > 5` and `NA * 0` are `NA`. State the general rule.

Solution [Supplement].

Step 1. (Rule.) R returns a known value exactly when the result is the same for every value the unknown could take; otherwise the result is again unknown.

Step 2. (Known results.) For every real u (including $\pm\infty$ and `NaN` in R's arithmetic) $u^0 = 1$, so `NA^0` is 1. For a logical u, $u \vee \text{TRUE} = \text{TRUE}$ and $\text{FALSE} \wedge u = \text{FALSE}$ whatever u is.

Step 3. (Unknown results.) `NA > 5` is `TRUE` for $u = 6$ and `FALSE` for $u = 4$, so it stays unknown. For `NA * 0`, $u \cdot 0 = 0$ for every finite u, but in floating-point arithmetic $\infty \cdot 0$ is `NaN`; since the missing number could be infinite, the result is not determined and R returns `NA`. This is the "tricky counterexample": the rule refers to all values R can represent, not only finite ones. ■

Exercise 3.3 (combining grouped means)

Split n rows into groups $g = 1, \dots, G$ of sizes n_g with group means $\bar{y}_g$ of a variable y.
(a) Show that the overall mean is $\bar{y} = \sum_g (n_g/n)\, \bar{y}_g$, so that the unweighted mean of the group means equals $\bar{y}$ when all n_g are equal or all $\bar{y}_g$ are equal, but not in general.
(b) Two groups A and B are each split by a binary variable s. In A, $s = 1$ for 80 of 100 rows with mean 10 and $s = 0$ for 20 rows with mean 2; in B, $s = 1$ for 20 of 100 rows with mean 11 and $s = 0$ for 80 rows with mean 3. Compare A and B within each level of s and overall.

Solution [Supplement].

(a) The groups partition the rows, so $\sum_i y_i = \sum_g \sum_{i \in g} y_i = \sum_g n_g \bar{y}_g$; divide by $n = \sum_g n_g$. The unweighted mean of the group means is $\frac{1}{G} \sum_g \bar{y}_g$, and

$$\bar{y} - \frac{1}{G} \sum_g \bar{y}_g = \sum_g \Big(\frac{n_g}{n} - \frac{1}{G} \Big) \bar{y}_g,$$

which vanishes when every weight difference is 0 (equal sizes) or when $\bar{y}_g \equiv c$ (then the sum is $c \sum_g (n_g/n - 1/G) = 0$). In general it does not vanish.

(b) Within each level B is higher: $11 > 10$ for $s = 1$ and $3 > 2$ for $s = 0$. Overall, by (a), A has mean $0.8 \cdot 10 + 0.2 \cdot 2 = 8.4$ and B has mean $0.2 \cdot 11 + 0.8 \cdot 3 = 4.6$, so A is higher. The comparison reverses (Simpson's paradox) because the groups have very different shares of the high-mean level $s = 1$. This is why `group_by()` on the right variables can change the conclusion of a `summarize()`. ■